ADS T1 ASSIGNMENT REPORT

UCE2021507 - Rajasi Barapatre
UCE2021510 - Sanika Chavan
UCE2021547 - Ketaki Paranjape

APPLE PLAY STORE

IMPORT STATEMENTS

```
import zipfile
import os
from google.colab import files
from google.colab import drive
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
```

LOADING DATASET

```
! pip install -q kaggle
files.upload()
drive.mount('/content/drive',force_remount=True)
!cp '/content/kaggle.json' '/content'
os.environ['KAGGLE_CONFIG_DIR']="/content"
!chmod 600 /content/kaggle.json
!kaggle datasets download -d gauthamp10/apple-appstore-apps
zip_ref1=zipfile.ZipFile('/content/apple-appstore-apps.zip','r')
zip_ref1.extractall('/tmp')
zip_ref1.close()
os.listdir('/tmp')
data=pd.read_csv('/tmp/appleAppData.csv',encoding='latin')
```

ANALYSING DATA

data.shape
(1230376, 21)

data.head()

1. App Name - Name of the App
2. App Id - An unique ID for each Application
3. Primary Genre - Each application belongs to certain category like Education, Productivity etc.
4. Average_User_Rating - The average rating given to the application by the users.
5. Reviews - The number of users who have given the rating.
6. Free - Boolean column which tells if an application is paid or free.
7. Price - Tells the price of the apps which are not free.
8. Size - The size of the application in bytes.
9. Developer Id - The Id of the developer who developed the app.
10. Developer Name - Name of the developer.
11. Released - The date the app was released.
12. Updated - The date the application was last updated.
13. Content Rating - The stratas of users.
14. Currency - The currency of the Price(USD)
15. Version - The version of the app.
16. AppStore_Url - The url of the appstore
17. Required_iOS_Version:- The required version of iOS needed to download the application.
18. Developer Website - The website of the developer who developed the app.
19. Developer Email - Email address of the developer.
20. Current_Version_Score - The current version required for the application.
21. Current_Version_Reviews - The current reviews given to the application

CHECKING NULL VALUES IN A DATASET

```
data.isnull().sum().sort_values(ascending = False)
```

```
Developer_Website 643988
Developer_Url 1109
Price  490
Size_Bytes  224
Released 3
App_Name 1
Free  0
Current_Version_Score 0
Reviews   0
Average_User_Rating 0
Developer 0
DeveloperId   0
App_Id  0
Currency  0
Version  0
Updated  0
Required_IOS_Version 0
Content_Rating 0
Primary_Genre   0
AppStore_Url   0
Current_Version_Reviews   0
dtype: int64
```

Developer Website has the highest number of null values, followed by Developer Url and Price

CHECKING DATA TYPES

```
data.dtypes
App_Id                  object
App_Name                object
AppStore_Url            object
Primary_Genre           object
Content_Rating          object
Size_Bytes             float64
Required_IOS_Version    object
Released                object
Updated                 object
Version                 object
Price                  float64
```

```
Currency                  object
Free                      bool
DeveloperId               int64
Developer                 object
Developer_Url             object
Developer_Website         object
Average_User_Rating       float64
Reviews                   int64
Current_Version_Score     float64
Current_Version_Reviews   int64
dtype: object
```

The dataset consists of 7 numeric columns, 1 boolean columns and the rest are of object data type (class type is string).

```
missing_percentage = (data.isnull().sum().sort_values(ascending = False)/len(data))*100
Missing_percentage
```

```
Developer_Website         52.340748
Developer_Url              0.090135
Price                      0.039825
Size_Bytes                 0.018206
Released                   0.000244
App_Name                   0.000081
Free                       0.000000
Current_Version_Score      0.000000
Reviews                    0.000000
Average_User_Rating        0.000000
Developer                  0.000000
DeveloperId                0.000000
App_Id                     0.000000
Currency                   0.000000
Version                    0.000000
Updated                    0.000000
Required_IOS_Version       0.000000
Content_Rating             0.000000
Primary_Genre              0.000000
AppStore_Url               0.000000
Current_Version_Reviews    0.000000
dtype: float64
```

Maximum null percentage in Developer_Website

CHECKING UNIQUE VALUES AND THEIR COUNTS

```
data['Free'].unique()
array([ True, False])

data['Primary_Genre'].value_counts()
Games                  193749
Business               128297
Education              119174
Utilities               94007
Lifestyle               84565
Food & Drink            73157
Health & Fitness        60724
Productivity            59430
Entertainment           53147
Shopping                49537
Finance                 44984
Travel                  43726
Sports                  31455
Music                   29898
Medical                 27995
Photo & Video           25957
Social Networking       25429
News                    20423
Reference               18615
Navigation              12636
Stickers                12407
Book                    11472
Weather                  3563
Graphics & Design        2781
Developer Tools          1951
Magazines & Newspapers   1297
Name: Primary_Genre, dtype: int64


data['Currency'].unique()
array(['USD'], dtype=object)
```

REMOVING IRRELEVANT DATA

```
data=data.drop('Developer_Url',axis=1)
data=data.drop('Developer_Website',axis=1)
data=data.drop('AppStore_Url',axis=1)
data=data.drop('Currency',axis=1)
```

1. Developer Website - Since this attribute has most missing values and is also irrelevant to further analysis of the dataset.
2. Developer Url - This column had the second most missing values and is irrelevant to the analysis of the application.
3. Appstore Url - This column is irrelevant to the analysis of the application.
4. Currency - This column has only USD as the value hence can be dropped

```
data.head()
```

	App_Id	App_Name	Primary_Genre	Content_Rating	Size_Bytes	Required_IOS_Version	Released	Updated	Version	Price	Free	DeveloperId	Developer	Average_User_Rating	Reviews	Current_Version_Score	Current_Version_Reviews
0	com.hkbu.arc.apaper	A+ Paper Guide	Education	4+	21993472.0	8.0	2017-09-28T03:02:41Z	2018-12-21T21:30:36Z	1.1.2	0.00	True	1375410542	HKBU ARC	0.0	0	0.0	0
1	com.dmitriev.abooks	A-Books	Book	4+	13135872.0	10.0	2015-08-31T19:31:32Z	2019-07-23T20:31:09Z	1.3	0.00	True	1031572001	Roman Dmitriev	5.0	1	5.0	1
2	no.terp.abooks	A-books	Book	4+	21943296.0	9.0	2021-04-14T07:00:00Z	2021-05-30T21:08:54Z	1.3.1	0.00	True	1457024163	Terp AS	0.0	0	0.0	0
3	fr.antoinettefleur.Book1	A-F Book #1	Book	4+	81851392.0	8.0	2012-02-10T03:40:07Z	2019-10-29T12:40:37Z	1.2	2.99	False	439568839	i-edlteur.com	0.0	0	0.0	0
4	com.imonstersoft.azdictionaryios	A-Z Synonyms Dictionary	Reference	4+	64692224.0	9.0	2020-12-16T08:00:00Z	2020-12-18T21:36:11Z	1.0.1	0.00	True	656731821	Ngov chiheang	0.0	0	0.0	0

DATA TYPE CONVERSION

```
data['Version'] = data['Version'].astype(str)
data.dtypes
App_Id                  object
App_Name                object
Primary_Genre           object
Content_Rating          object
Size_Bytes             float64
Required_IOS_Version    object
Released                object
Updated                 object
Version                 object
Price                  float64
Free                      bool
DeveloperId              int64
Developer               object
Average_User_Rating    float64
Reviews                  int64
Current_Version_Score  float64
Current_Version_Reviews  int64
dtype: object
```

```
unique_data_types = data['Version'].apply(type).unique()
print(unique_data_types)
```

```
[<class 'str'>]
```

```
data['Version'] = data['Version'].astype(str)
data.dtypes
```

```
App_Id                  object
App_Name                object
Primary_Genre           object
Content_Rating          object
Size_Bytes             float64
Required_IOS_Version    object
Released                object
Updated                 object
Version                 object
Price                  float64
Free                    bool
DeveloperId            int64
Developer               object
Average_User_Rating    float64
Reviews                 int64
Current_Version_Score  float64
Current_Version_Reviews  int64
dtype: object
```

Converted the Version Column as a string

CHECKING FOR DUPLICATES

```
has_duplicate_rows = data.duplicated().any()

# Print the result
print(has_duplicate_rows)
```

```
False
```

No duplicate rows

ADJUSTING DATA

```
data['ContentRating'] = data['Content_Rating'].str.replace('+', '', regex=False)
data.drop(columns=['Content_Rating'], inplace=True)
```

CREATION OF NEW VARIABLES

```
data['Released_Year'] = data['Released'].str[:4]
data['Updated_Year'] = data['Updated'].str[:4]
data.drop(columns=['Released'], inplace=True)
data.drop(columns=['Updated'], inplace=True)
```

The date as a whole is not required, as only the year can be extracted and suitable analysis can be performed on it.

```
data.head()
```

	App_Id	App_Name	Primary_Genre	Size_Bytes	Required_IOS_Version	Version	Price	Free	DeveloperId	Developer	Average_User_Rating	Reviews	Current_Version_Score	Current_Version_Reviews	ContentRating	Released_Year	Updated_Year
0	com.hkbu.arc.apaper	A+ Paper Guide	Education	21993472.0	8.0	1.1.2	0.00	True	1375410542	HKBU ARC	0.0	0	0.0	0	4	2017	2018
1	com.dmitriev.abooks	A-Books	Book	13135872.0	10.0	1.3	0.00	True	1031572001	Roman Dmitriev	5.0	1	5.0	1	4	2015	2019
2	no.terp.abooks	A-books	Book	21943296.0	9.0	1.3.1	0.00	True	1457024163	Terp AS	0.0	0	0.0	0	4	2021	2021
3	fr.antoinettefleur.Book1	A-F Book #1	Book	81851392.0	8.0	1.2	2.99	False	439568839	i-editeur.com	0.0	0	0.0	0	4	2012	2019
4	com.imonstersoft.azdictionaryios	A-Z Synonyms Dictionary	Reference	64692224.0	9.0	1.0.1	0.00	True	656731821	Ngov chiheang	0.0	0	0.0	0	4	2020	2020

```
free={
    True:1,
    False:0
}
data['Free']= data['Free'].map(free)
```

Mapped the values of True and False in the boolean columns to 1 and 0 so as to make computation easier.

FILLING MISSING VALUES

```
mean_price = data['Price'][~data['Price'].isna()].mean()
data['Price'] = np.where((data['Free'] == 0) & (data['Price'].isna()), mean_price, data['Price'])
data['Price'] = np.where((data['Free'] == 1) & (data['Price'].isna()), 0, data['Price'])
```

Replacing the null values of the Price column by the mean

```
data.dropna(subset=['App_Name', 'Released_Year'], inplace=True)
```

Dropping the rows of App_Name and Released_Year as only 1 and 3 values are missing respectively. Hence dropping these rows wont affect the dataset

```
mean_size = data['Size_Bytes'][~data['Size_Bytes'].isna()].mean()
data['Size_Bytes'].fillna(mean_size, inplace=True)
```

The null values of Size is filled with the mean

```
data.isnull().sum().sort_values(ascending = False)
```

```
App_Id                  0
Developer               0
Released_Year           0
ContentRating           0
Current_Version_Reviews    0
Current_Version_Score      0
Reviews                 0
Average_User_Rating        0
DeveloperId             0
App_Name                0
Free                    0
Price                   0
Version                 0
Required_IOS_Version       0
Size_Bytes              0
Primary_Genre           0
Updated_Year            0
dtype: int64
```

```
rep = data['Required_IOS_Version'].min()
data['Required_IOS_Version'].fillna(rep , inplace=True)
```

```
data['Required_IOS_Version'].isna().sum()
0
```

```
data.isna().sum()
App_Id                     0
App_Name                   0
Primary_Genre              0
Size_Bytes                 0
Required_IOS_Version       0
Version                    0
Price                      0
Free                       0
DeveloperId                0
Developer                  0
Average_User_Rating        0
Reviews                    0
Current_Version_Score      0
Current_Version_Reviews    0
ContentRating              0
Released_Year              0
Updated_Year               0
dtype: int64
```

data

	App_Id	App_Name	Primary_Genre	Size_Bytes	Required_IOS_Version	Version	Price	Free	DeveloperId	Developer	Average_User_Rating	Reviews	Current_Version_Score	Current_Version_Reviews	ContentRating	Released_Year	Updated_Year
0	com.hkbu.arc.apaper	A+ Paper Guide	Education	21993472.0	8.0	1.1.2	0.00	1	1375410542	HKBU ARC	0.00000	0	0.00000	0	4	2017	2018
1	com.dmitriev.abooks	A-Books	Book	13135872.0	10.0	1.3	0.00	1	1031572001	Roman Dmitriev	5.00000	1	5.00000	1	4	2015	2019
2	no.terp.abooks	A-books	Book	21943296.0	9.0	1.3.1	0.00	1	1457024163	Terp AS	0.00000	0	0.00000	0	4	2021	2021
3	fr.antoinetteleur.Book1	A-F Book #1	Book	81851392.0	8.0	1.2	2.99	0	439568839	i-editeur.com	0.00000	0	0.00000	0	4	2012	2019
4	com.imonstersoft.azdictionaryios	A-Z Synonyms Dictionary	Reference	64692224.0	9.0	1.0.1	0.00	1	656731821	Ngov chiheang	0.00000	0	0.00000	0	4	2020	2020
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1230371	com.ledtech.sadblock	SĐµÑ□urity Đ□dBlß¾Ñ□ikĐµr	Utilities	16666624.0	13.0	1.0.1	0.00	1	1522287989	LED-TECHNOLOGIES	3.91608	143	3.91608	143	4	2020	2020
1230372	com.securex.vpn	SĐµÑ□urĐµĐ¥ VĐ N - Wifi Proxy	Utilities	39016448.0	9.0	1.1	0.00	1	1492288123	Trust VPN Ltd.	4.82733	1500	4.82733	1500	4	2019	2020
1230373	com.beelab.SoTayXayDung	Sá»□ tay XÃ¢y dá»±ng	Utilities	17223680.0	9.0	1.0	0.00	1	1438594214	Luu Minh	4.00000	1	4.00000	1	4	2018	2018
1230374	com.icc.sttb	Sá»□ tay Ä□á°£ng viÃªn ThÃ¡i BÃ¬nh	Utilities	56716288.0	10.0	1.2.5	0.00	1	1515469508	ThÃ¡i BÃ¬nh	0.00000	0	0.00000	0	4	2021	2021
1230375	com.vnptlonganios.sodiemthongminh	Sá»□ Ä□iá»□m ThÃ ng Minh	Utilities	85135360.0	8.0	1.9	0.00	1	1350355912	Pham Thanh Vo	0.00000	0	0.00000	0	4	2018	2019

1230372 rows × 17 columns

DATA ANALYSIS

```
developer_counts = data['DeveloperId'].value_counts()
print(developer_counts)

developer_counts1 = data['Developer'].value_counts()
print(developer_counts1)
```

```
553996348     3966
991472673     1649
1476391440    1506
472317932     1247
623199517     1131
              ...
929955962        1
549094179        1
1162182706       1
1111380763       1
1438594214       1
Name: DeveloperId, Length: 509285, dtype: int64
ChowNow                      3966
Touch2Success                1649
Alexander Velimirovic        1506
MINDBODY, Incorporated       1247
Phorest                      1131
                             ...
DZ BANK AG                      1
Alpha Mortgage Corporation      1
RE/MAX Alliance Colorado        1
ALE SOLUTIONS, INC.             1
Luu Minh                        1
Name: Developer, Length: 505255, dtype: int64
```

UNIVARIATE ANALYSIS

```
max_price = data['Price'].max()
second_min_price = data['Price'].nsmallest(2).iloc[-1]
print("Maximum Price:", max_price)
print("Minimum Price:", second_min_price)
```

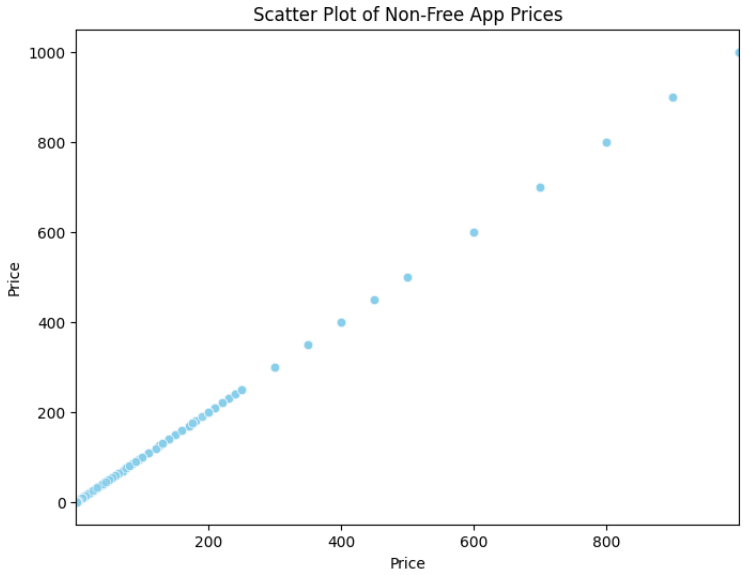
Maximum Price: 999.99
Minimum Price: 0.0

- SCATTER PLOT FOR NON FREE APP PRICES

```
non_free_apps = data[data['Price'] > 0.0]

max_price = non_free_apps['Price'].max()
min_price = non_free_apps['Price'].min()

plt.figure(figsize=(8, 6))
sns.scatterplot(data=non_free_apps, x='Price', y='Price', color='skyblue',
alpha=0.7)
plt.xlabel('Price')
plt.ylabel('Price')
plt.title('Scatter Plot of Non-Free App Prices')
plt.xlim(min_price, max_price)
plt.show()
```

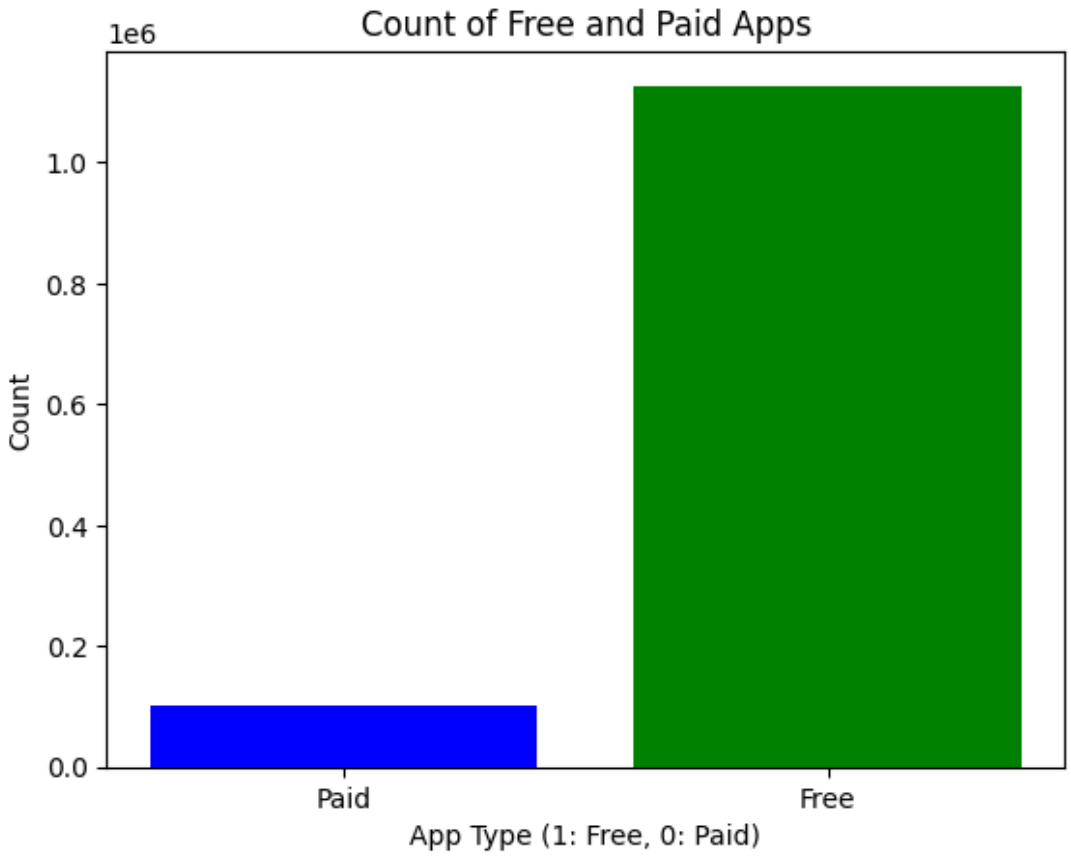


Most of the non free apps lies in range 0 to approx 300 as they are more clustered

- Count the number of free and paid apps

```
app_counts = data['Free'].value_counts()
print(app_counts)
plt.bar(app_counts.index, app_counts.values, color=['green', 'blue'])
plt.xlabel('App Type (1: Free, 0: Paid)')
plt.ylabel('Count')
plt.title('Count of Free and Paid Apps')
plt.xticks(app_counts.index, ['Free', 'Paid'])
plt.show()
```

```
1 1127383
0 102989
Name: Free, dtype: int64
```



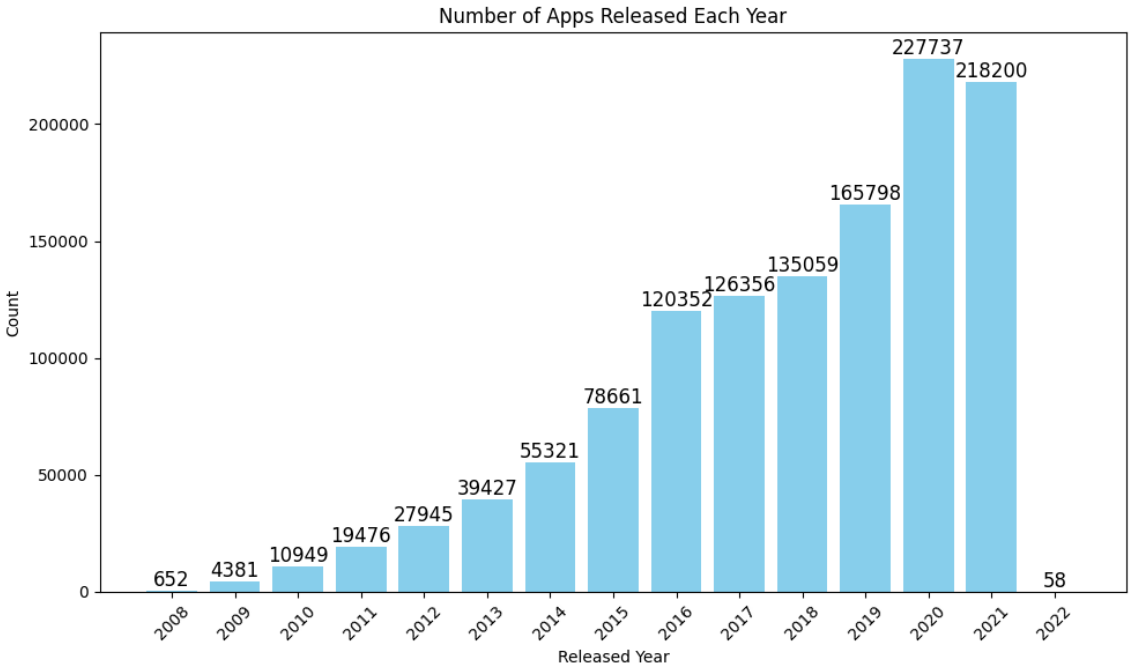
There are total 11,27,383 free apps and 1,02,989 are total paid apps.

- Count the number of apps released each year

```
# Year count
year_counts = data['Released_Year'].value_counts().sort_index()

plt.figure(figsize=(10, 6))
plt.bar(year_counts.index, year_counts.values, color='skyblue')
plt.xlabel('Released Year')
plt.ylabel('Count')
plt.title('Number of Apps Released Each Year')

#for the count over the bar plot
for i, count in enumerate(year_counts.values):
    plt.text(year_counts.index[i], count, str(count), ha='center', va='bottom',
fontsize=12)
#labels
plt.xticks(year_counts.index, rotation=45)
plt.tight_layout()
plt.show()
```



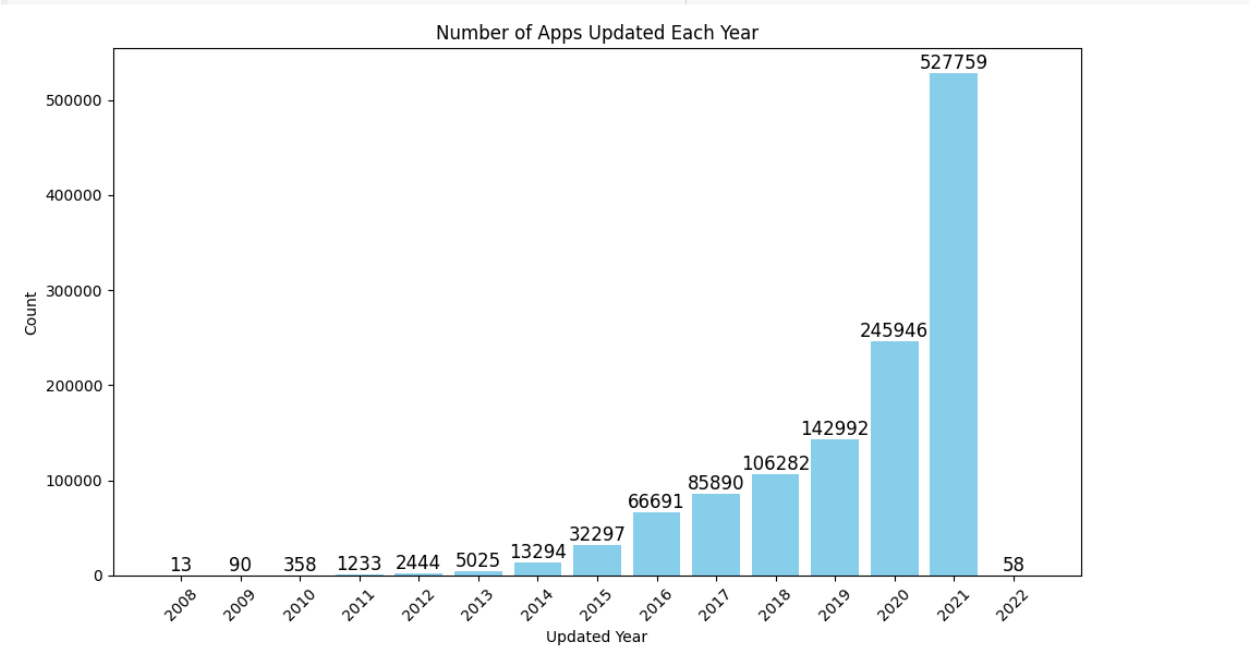
The maximum apps released are in year 2020(227737)
The minimum apps released are in year 2022(58)

- Count the number of apps updated in each year
  #Year count of the updated year
  year_counts = data['Updated_Year'].value_counts().sort_index()

  plt.figure(figsize=(10, 6))
  plt.bar(year_counts.index, year_counts.values, color='skyblue')
  plt.xlabel('Updated Year')
  plt.ylabel('Count')
  plt.title('Number of Apps Updated Each Year')

  # Display the count above each bar
  for i, count in enumerate(year_counts.values):
  plt.text(year_counts.index[i], count, str(count), ha='center', va='bottom', fontsize=12)

  plt.xticks(year_counts.index, rotation=45)
  plt.tight_layout()
  plt.show()



Maximum apps updated in year 2021(527759)
Minimum apps updated in year 2022(58)

- Pie chart of Top 10 Primary Genre

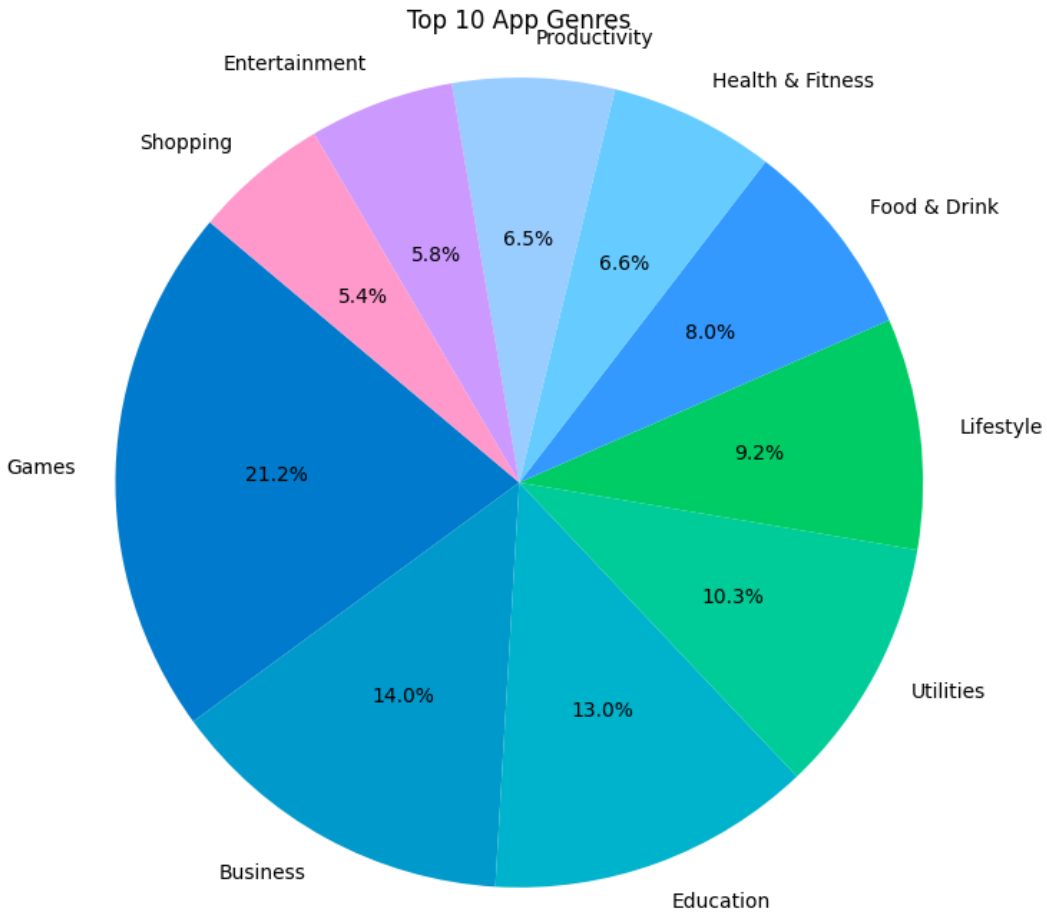
```
genre_counts = data['Primary_Genre'].value_counts()
print(genre_counts)
#count of each primary genre
top_10_genre_counts = genre_counts.head(10)
#for different shades of the pie chart
shades_of_blue = ['#007acc', '#0099cc', '#00b3cc', '#00cc99', '#00cc66',
'#3399ff', '#66ccff', '#99ccff', '#cc99ff', '#ff99cc']

plt.figure(figsize=(8, 8))
plt.pie(top_10_genre_counts, labels=top_10_genre_counts.index,
autopct='%1.1f%%', startangle=140, colors=shades_of_blue)
plt.title('Top 10 App Genres')
plt.axis('equal')

plt.show()
```

```
Games                193746
Business             128297
Education            119174
Utilities             94007
Lifestyle             84565
Food & Drink          73157
Health & Fitness      60724
Productivity          59430
Entertainment         53147
Shopping              49537
Finance               44984
Travel                43726
Sports                31455
Music                 29898
Medical               27995
Photo & Video         25957
Social Networking     25429
News                  20422
Reference             18615
Navigation            12636
Stickers              12407
Book                  11472
```

Weather 3563
Graphics & Design 2781
Developer Tools 1951
Magazines & Newspapers 1297
Name: Primary_Genre, dtype: int64



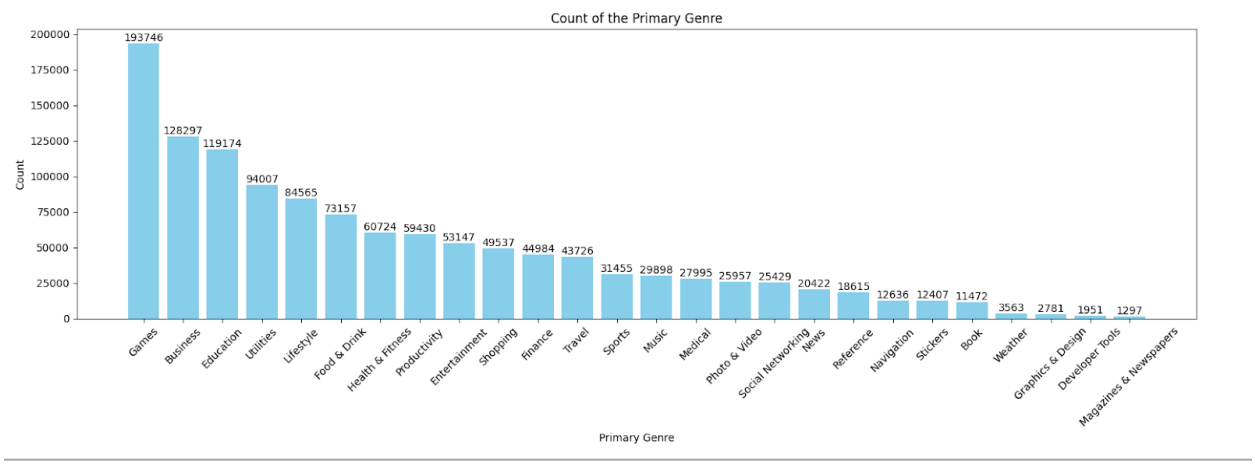
Maximum Primary_Genre app is Games and Minimum Primary_Genre app is Magazines & Newspapers

- Count of the Primary Genre

```
plt.figure(figsize=(16, 6))
plt.bar(genre_counts.index, genre_counts.values, color='skyblue')  #passing the genre count and the values to be plot
plt.xlabel('Primary Genre')
plt.ylabel('Count')
plt.title('Count of the Primary Genre')

for i, count in enumerate(genre_counts.values):
    plt.text(genre_counts.index[i], count, str(count), ha='center', va='bottom', fontsize=10)

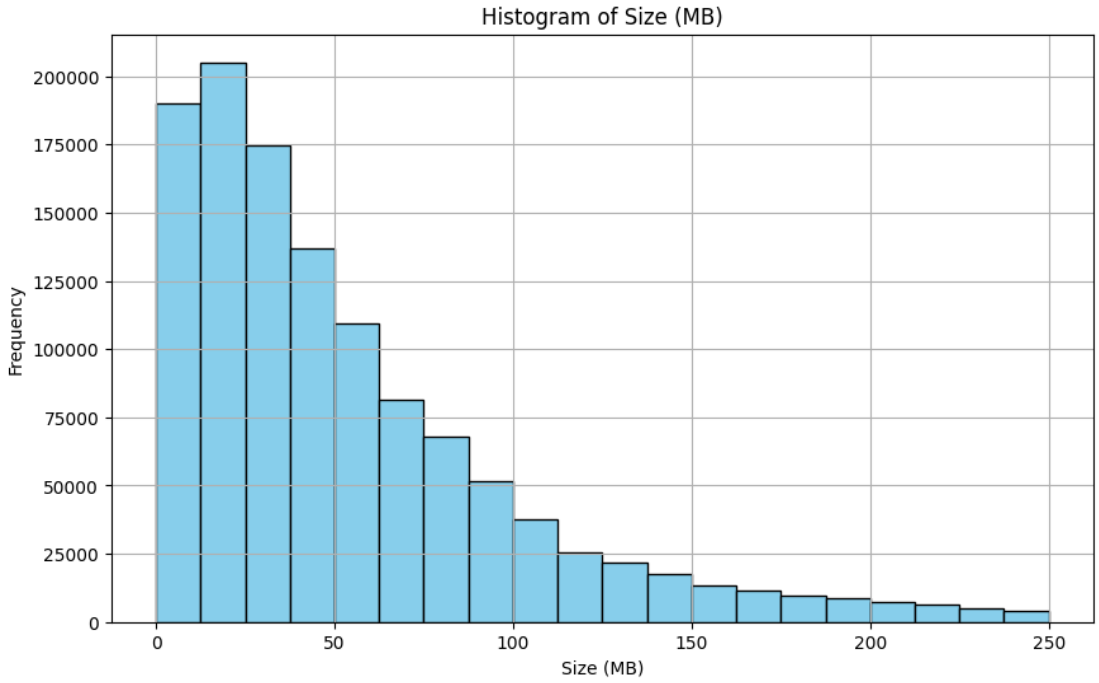
plt.xticks(genre_counts.index, rotation=45)
plt.tight_layout()
plt.show()
```




```
print(data['Size_Bytes'].max())
print(data['Size_Bytes'].min())
#Converting Size Bytes to MB
data['Size_Bytes']=data['Size_Bytes']/1024/1024
data.head()
```

App_Id	App_Name	Primary_Genre	Size_Bytes	Required_IOS_Version	Version	Price	Free	DeveloperId	Developer	Average_User_Rating	Reviews	Current_Version_Score	Current_Version_Reviews	ContentRating	Released_Year	Updated_Year
com.hkbu.arc.apaper	A+ Paper Guide	Education	20.974609	8.0	1.1.2	0.00	1	1375410542	HKBU ARC	0.0	0	0.0	0	4	2017	2018
com.dmitriev.abooks	A-Books	Book	12.527344	10.0	1.3	0.00	1	1031572001	Roman Dmitriev	5.0	1	5.0	1	4	2015	2019
no.terp.abooks	A-books	Book	20.926758	9.0	1.3.1	0.00	1	1457024163	Terp AS	0.0	0	0.0	0	4	2021	2021
fr.antoinetteleur.Book1	A-F Book #1	Book	78.059570	8.0	1.2	2.99	0	439568839	i-editeur.com	0.0	0	0.0	0	4	2012	2019
om.imonstersoft.azdictionaryios	A-Z Synonyms Dictionary	Reference	61.695312	9.0	1.0.1	0.00	1	656731821	Ngov chiheang	0.0	0	0.0	0	4	2020	2020

- Histogram plot of the Size(MB)
  plt.figure(figsize=(10, 6))
  #BINS AS 20 AND PRICE RANGE OF 250
  plt.hist(data['Size_Bytes'], bins=20, range=(0, 250), color='skyblue',
  edgecolor='black')
  plt.xlabel('Size (MB)')
  plt.ylabel('Frequency')
  plt.title('Histogram of Size (MB)')
  plt.grid(True)
  plt.show()

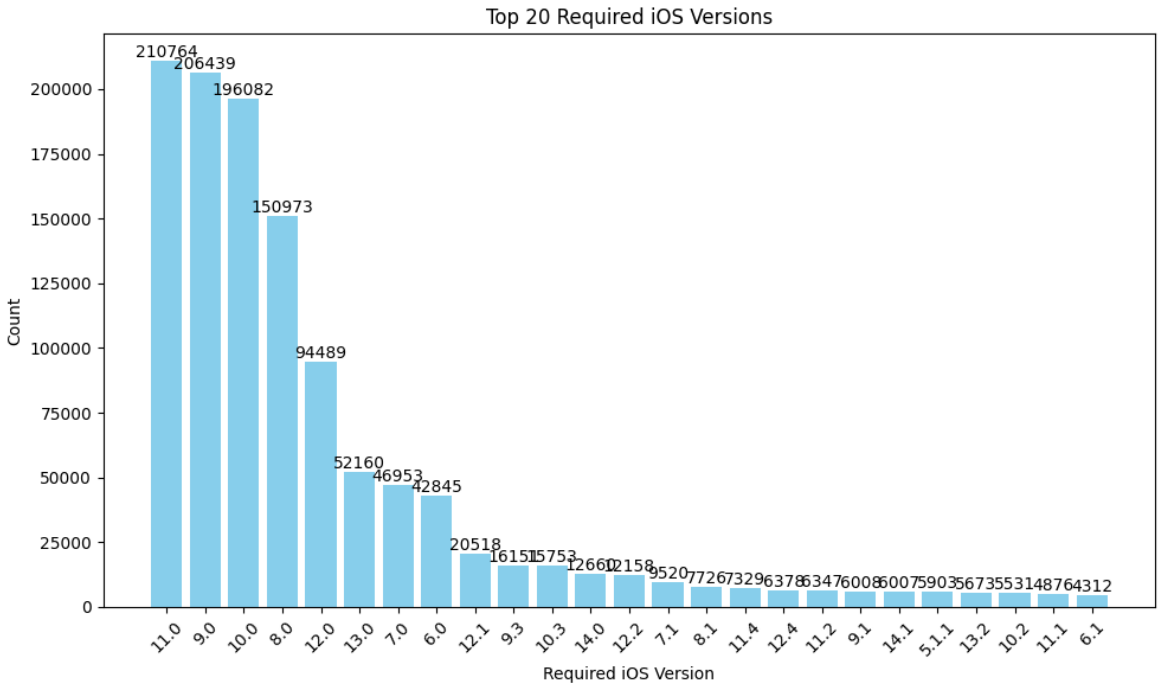


- The counts of each iOS version
  ios_version_counts = data['Required_IOS_Version'].value_counts().head(25)
  #Considering only the start 25 data
  plt.figure(figsize=(10, 6))
  bars = plt.bar(ios_version_counts.index, ios_version_counts.values,
  color='skyblue')
  plt.xlabel('Required iOS Version')
  plt.ylabel('Count')
  plt.title('Top 20 Required iOS Versions')

```
plt.xticks(rotation=45)

for i, count in enumerate(ios_version_counts ):
    plt.text(ios_version_counts .index[i], count, str(count), ha='center', va='bottom',
fontsize=10)

plt.tight_layout()
plt.show()
```



The maximum required iOS Version is 11.0 and Minimum is 6.1

Mapping the Content Rating

```
data["ContentRating"]=data["ContentRating"].replace("4","Toddler")
data["ContentRating"]=data["ContentRating"].replace("9","Children")
data["ContentRating"]=data["ContentRating"].replace("12","Teens")
data["ContentRating"]=data["ContentRating"].replace("17","Adults")
data["ContentRating"]=data["ContentRating"].replace("Not yet rated","Everyone")
```

```
agegroup = data['ContentRating'].unique()
custom_order = ['Toddler','Children', 'Teens', 'Adults']
sorted_array = sorted(['Toddler','Children', 'Adults', 'Teens'], key=lambda x:
custom_order.index(x))
print(sorted_array)
```

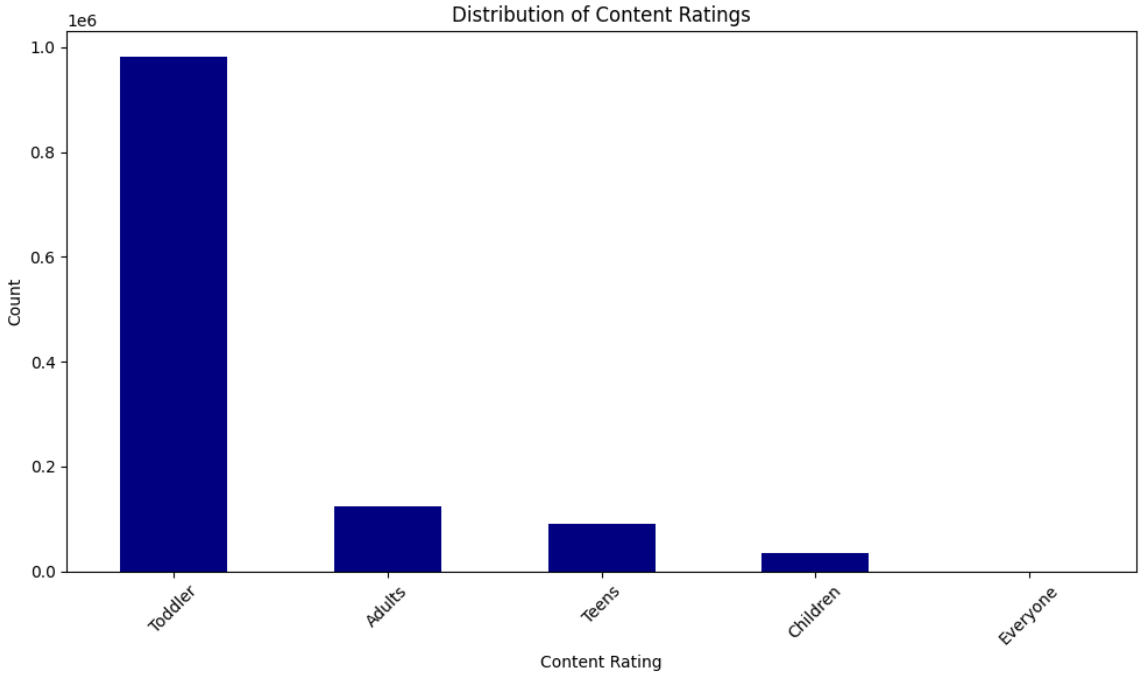
```
['Toddler', 'Children', 'Teens', 'Adults']
```

App_Id	App_Name	Primary_Genre	Size_Bytes	Required_IOS_Version	Version	Price	Free	DeveloperId	Developer	Average_User_Rating	Reviews	Current_Version_Score	Current_Version_Reviews	ContentRating	Released_Year	Updated_Year
rc.apaper	A+ Paper Guide	Education	20.974609	8.0	1.1.2	0.00	1	1375410542	HKBU ARC	0.0	0	0.0	0	Toddler	2017	2018
ev.abooks	A-Books	Book	12.527344	10.0	1.3	0.00	1	1031572001	Roman Dmitriev	5.0	1	5.0	1	Toddler	2015	2019
rp.abooks	A-books	Book	20.926758	9.0	1.3.1	0.00	1	1457024163	Terp AS	0.0	0	0.0	0	Toddler	2021	2021
eur.Book1	A-F Book #1	Book	78.059570	8.0	1.2	2.99	0	439568839	i-editeur.com	0.0	0	0.0	0	Toddler	2012	2019
tionaryios	A-Z Synonyms Dictionary	Reference	61.695312	9.0	1.0.1	0.00	1	656731821	Ngov chiheang	0.0	0	0.0	0	Toddler	2020	2020

- Distribution of Content Ratings

```
content_rating_counts = data['ContentRating'].value_counts()
print(content_rating_counts)
plt.figure(figsize=(10, 6))
content_rating_counts.plot(kind='bar', color='navy')
plt.xlabel('Content Rating')
plt.ylabel('Count')
plt.title('Distribution of Content Ratings')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

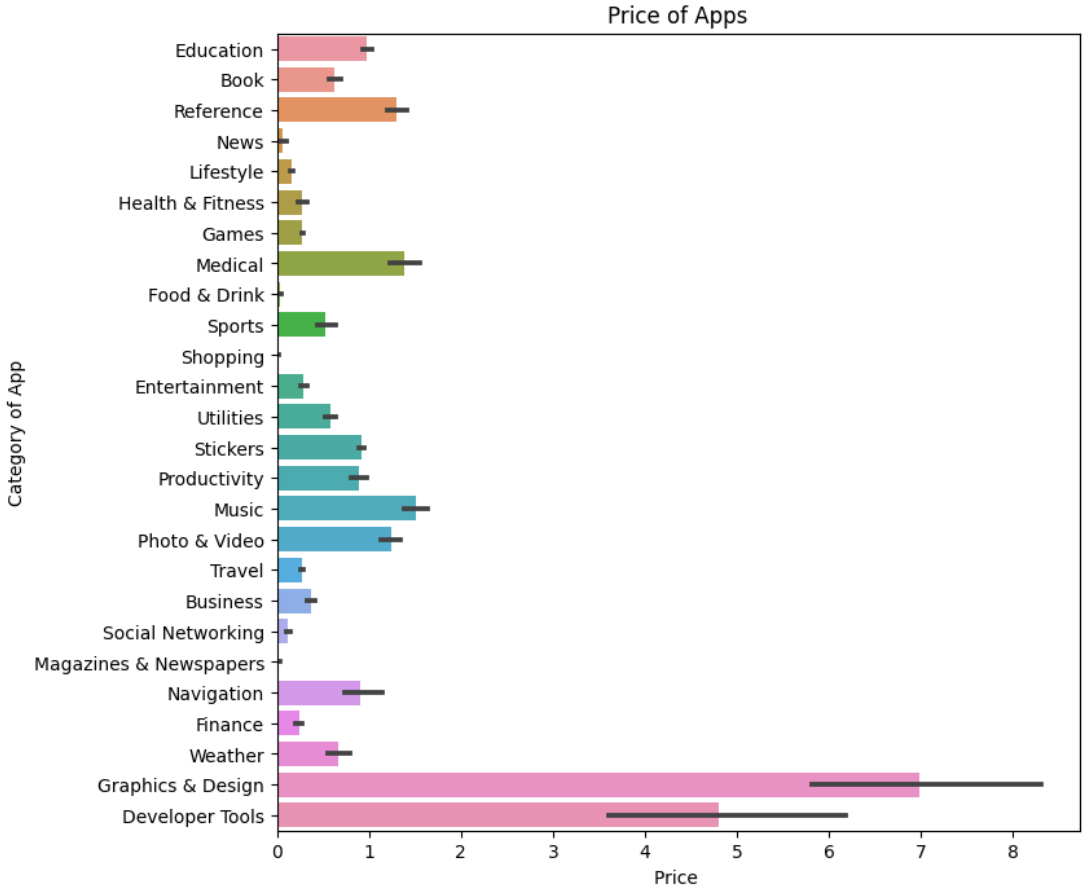
```
Toddler 980969
Adults 124127
Teens 89569
Children 35697
Everyone 10
Name: ContentRating, dtype: int64
```



Maximum count is Toddler(980969)
Minimum is Everyone(10)

BIVARIATE ANALYSIS

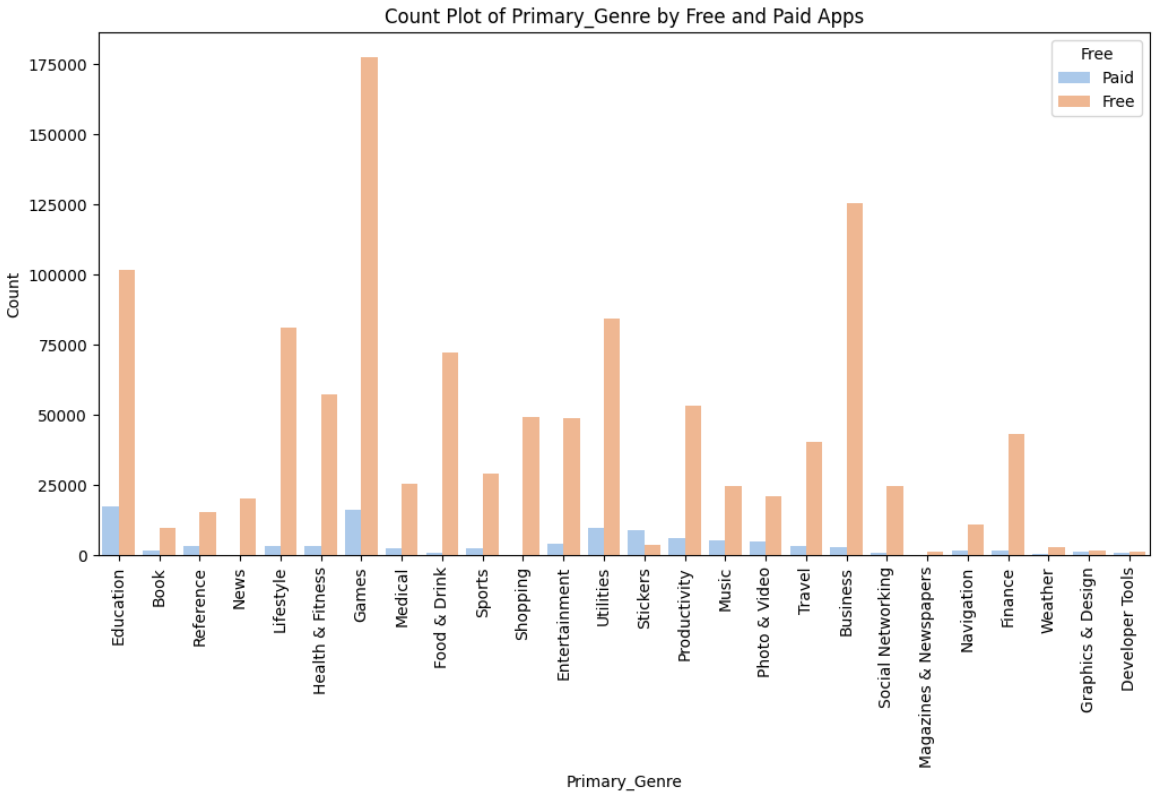
- Which primary genre has the highest price on AppleApps?
  plt.figure(figsize=(8,8))
  sns.barplot(data, x="Price", y="Primary_Genre")
  plt.xlabel('Price ')
  plt.ylabel('Category of App')
  plt.title('Price of Apps')



Graphics and Design have the highest price

- Count of Primary Genre whether they are free or paid
  plt.figure(figsize=(12, 6))
  sns.countplot(x='Primary_Genre', hue='Free', data=data, palette='pastel')
  plt.xlabel('Primary_Genre')
  plt.ylabel('Count')

```
plt.title('Count Plot of Primary_Genre by Free and Paid Apps')
plt.xticks(rotation=90)
plt.legend(title='Free', loc='upper right', labels=['Paid', 'Free'])
plt.show()
```

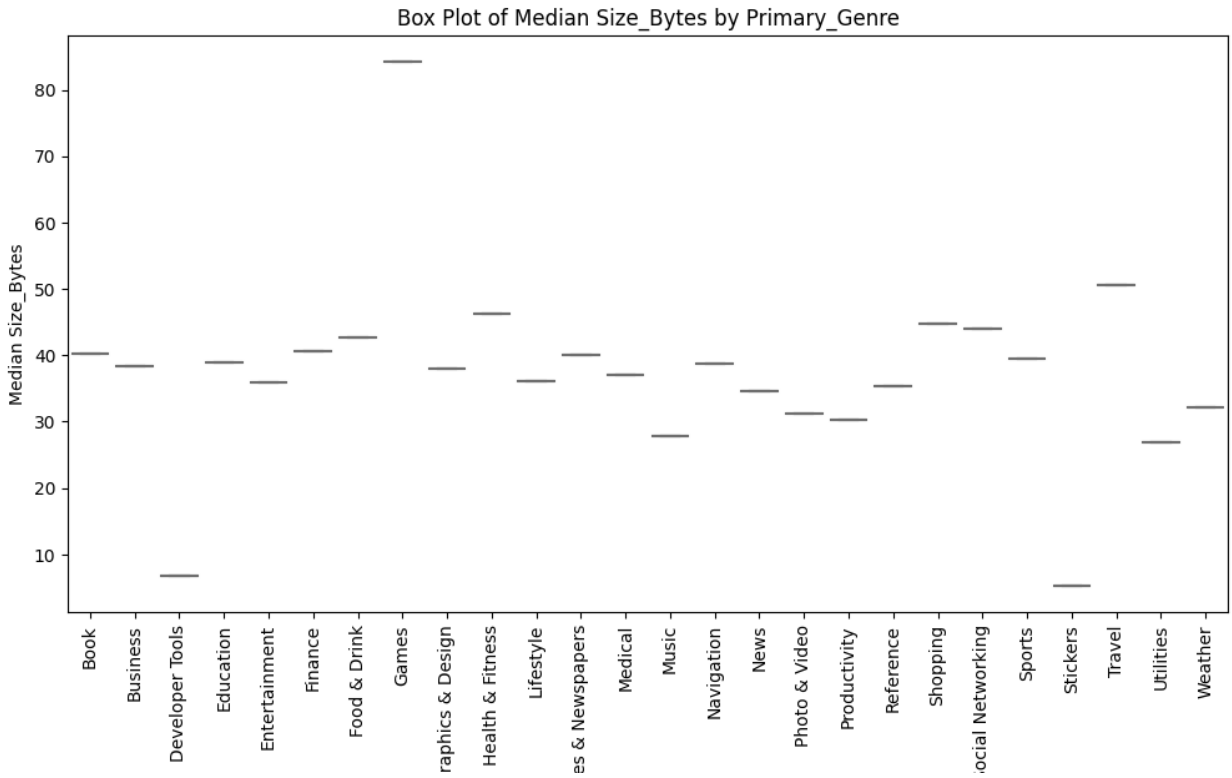


Games are the most Free apps in the dataset

- Median of the Size of each Genre
  median_size_by_genre = data.groupby('Primary_Genre')['Size_Bytes'].median().reset_index()
  print(median_size_by_genre)
  plt.figure(figsize=(12, 6))
  sns.boxplot(x='Primary_Genre', y='Size_Bytes', data=median_size_by_genre, palette='pastel') #passing the median size of the genre and plotting the box plot
  plt.xlabel('Primary_Genre')
  plt.ylabel('Median Size_Bytes')

```
plt.title('Box Plot of Median Size_Bytes by Primary_Genre')
plt.xticks(rotation=90)
plt.show()
```

	Primary_Genre	Size_Bytes
0	Book	40.268066
1	Business	38.425781
2	Developer Tools	6.728115
3	Education	38.937012
4	Entertainment	35.903320
5	Finance	40.662109
6	Food & Drink	42.727539
7	Games	84.201660
8	Graphics & Design	37.988281
9	Health & Fitness	46.277344
10	Lifestyle	36.033203
11	Magazines & Newspapers	40.030273
12	Medical	37.021484
13	Music	27.817871
14	Navigation	38.820312
15	News	34.655273
16	Photo & Video	31.294922
17	Productivity	30.270996
18	Reference	35.330078
19	Shopping	44.742188
20	Social Networking	44.063477
21	Sports	39.443359
22	Stickers	5.237305
23	Travel	50.532715
24	Utilities	26.806641
25	Weather	32.199219




```
#Converting the Required ios version to numeric
data['Required_IOS_Version'] = pd.to_numeric(data['Required_IOS_Version'],
errors='coerce')

max_ios_version_by_genre =
data.groupby('Primary_Genre')['Required_IOS_Version'].max().reset_index()
#grouping the primary genre with the required ios version

print(max_ios_version_by_genre)
```

	Primary_Genre	Required_IOS_Version
0	Book	15.0
1	Business	15.0
2	Developer Tools	15.0
3	Education	15.0
4	Entertainment	15.0
5	Finance	15.0
6	Food & Drink	15.0
7	Games	15.0
8	Graphics & Design	15.0
9	Health & Fitness	15.0

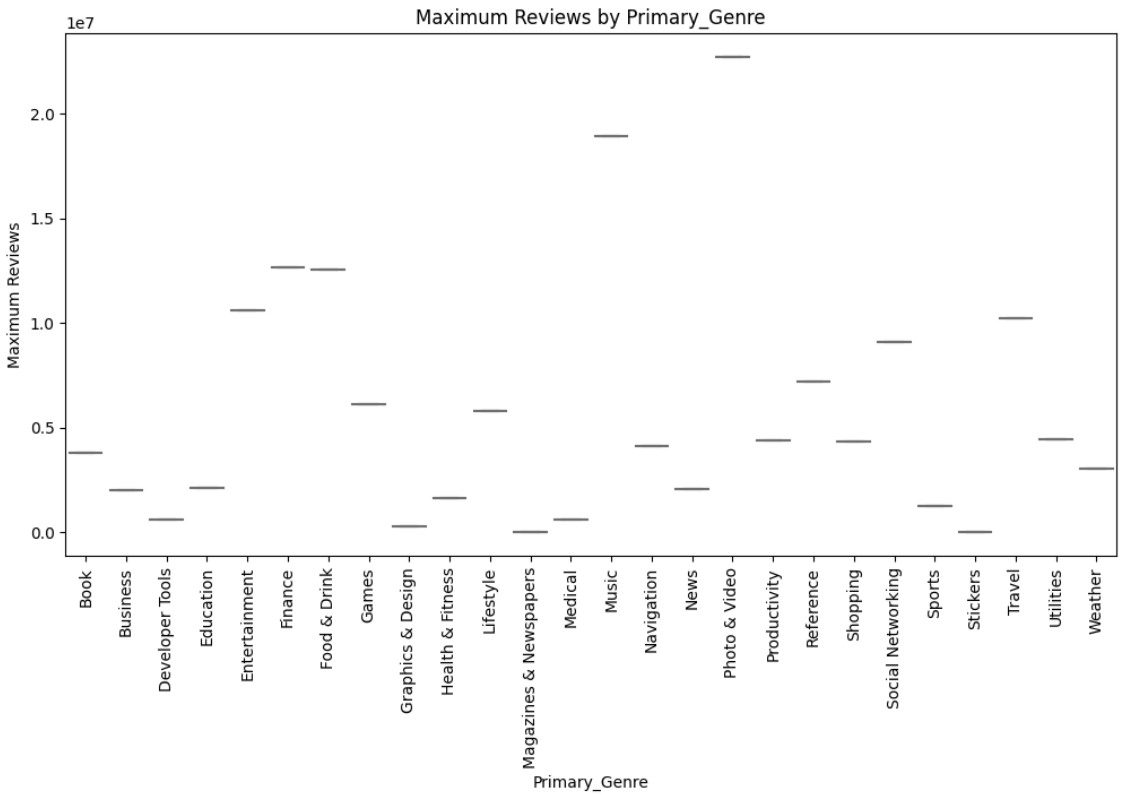
10	Lifestyle	15.0
11	Magazines & Newspapers	15.0
12	Medical	15.0
13	Music	15.0
14	Navigation	15.0
15	News	15.0
16	Photo & Video	15.0
17	Productivity	15.0
18	Reference	15.0
19	Shopping	15.0
20	Social Networking	15.0
21	Sports	15.0
22	Stickers	14.5
23	Travel	15.0
24	Utilities	15.0
25	Weather	15.0

- The maximum review count for each Primary_Genre

```
max_reviews_by_genre =
data.groupby('Primary_Genre')['Reviews'].max().reset_index()   #Grouping
the review with the Primary Genre
print(max_reviews_by_genre)
plt.figure(figsize=(12, 6))
sns.boxplot(x='Primary_Genre', y='Reviews',
data=max_reviews_by_genre, palette='pastel')
plt.xlabel('Primary_Genre')
plt.ylabel('Maximum Reviews')
plt.title('Maximum Reviews by Primary_Genre')
plt.xticks(rotation=90)
plt.show()
```

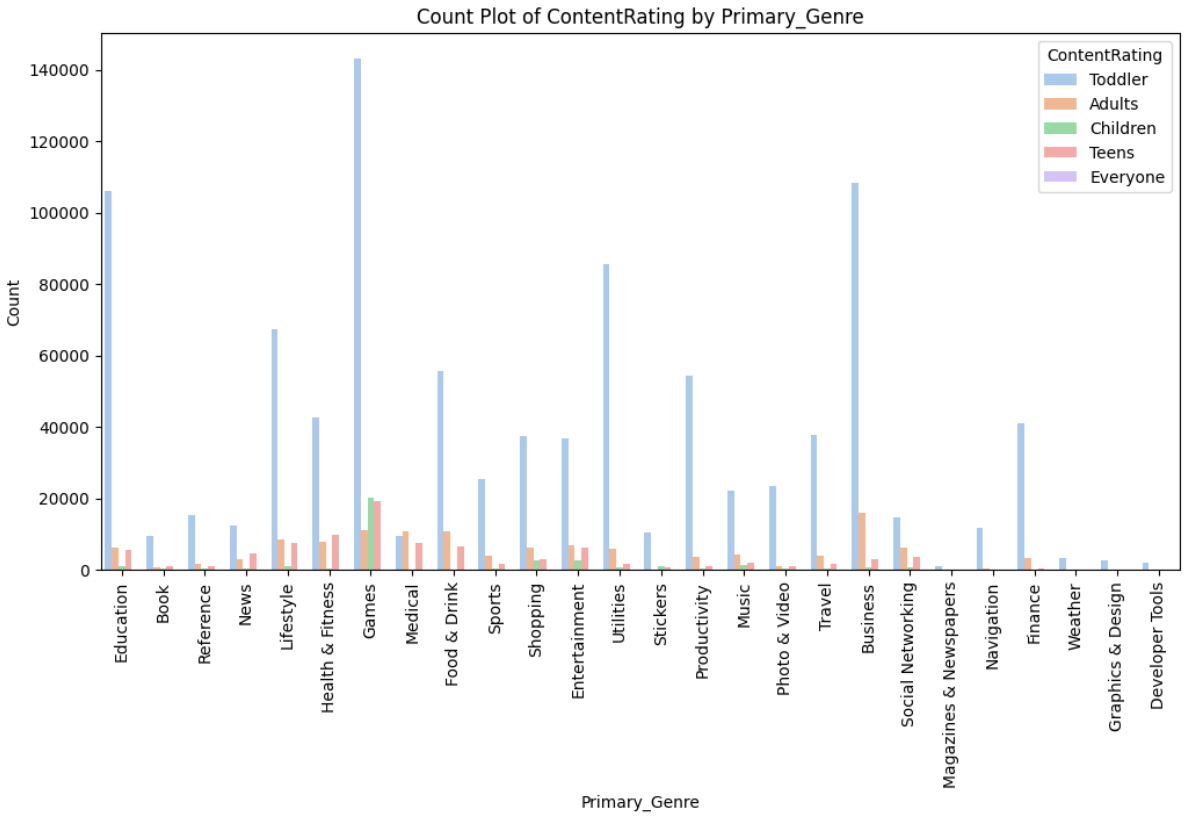
	Primary_Genre	Reviews
0	Book	3769815
1	Business	2040151
2	Developer Tools	613670
3	Education	2147317
4	Entertainment	10598509
5	Finance	12634191
6	Food & Drink	12517538

7	Games	6127588
8	Graphics & Design	306916
9	Health & Fitness	1658070
10	Lifestyle	5771436
11	Magazines & Newspapers	12547
12	Medical	584872
13	Music	18893225
14	Navigation	4132208
15	News	2079279
16	Photo & Video	22685334
17	Productivity	4371475
18	Reference	7181127
19	Shopping	4349737
20	Social Networking	9090956
21	Sports	1274874
22	Stickers	4105
23	Travel	10241777
24	Utilities	4425434
25	Weather	3026906



Photos and Video have the maximum reviews

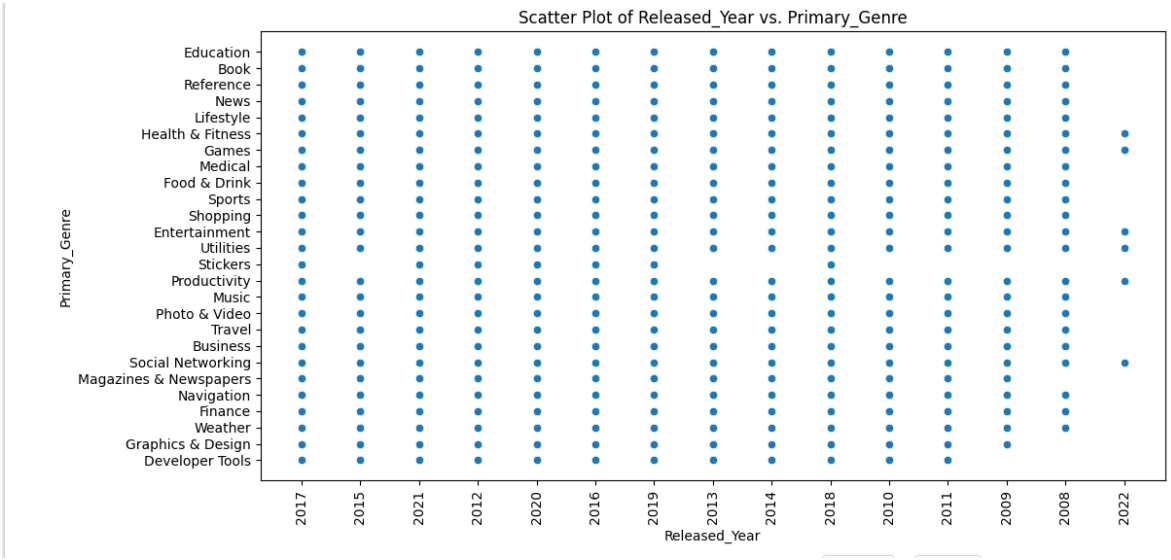
- Count of each Primary_Genre with the Content Rating
  plt.figure(figsize=(12, 6))
  sns.countplot(x='Primary_Genre', hue='ContentRating', data=data, palette='pastel')
  plt.xlabel('Primary_Genre')
  plt.ylabel('Count')
  plt.title('Count Plot of ContentRating by Primary_Genre')
  plt.xticks(rotation=90)
  plt.legend(title='ContentRating', loc='upper right')
  plt.show()



Toddler have the highest count in each primary Genre

- Scatter Plot of the Released Year

```
plt.figure(figsize=(12, 6))
sns.scatterplot(x='Released_Year', y='Primary_Genre', data=data,
palette='pastel')
plt.xlabel('Released_Year')
plt.ylabel('Primary_Genre')
plt.title('Scatter Plot of Released_Year vs. Primary_Genre')
plt.xticks(rotation=90)
plt.show()
```




```
#for verification
education_apps = data[data['Primary_Genre'] == 'Education']

education_release_years = education_apps['Released_Year'].unique()

print("Years in which 'Education' primary genre apps were released:",
education_release_years)
```

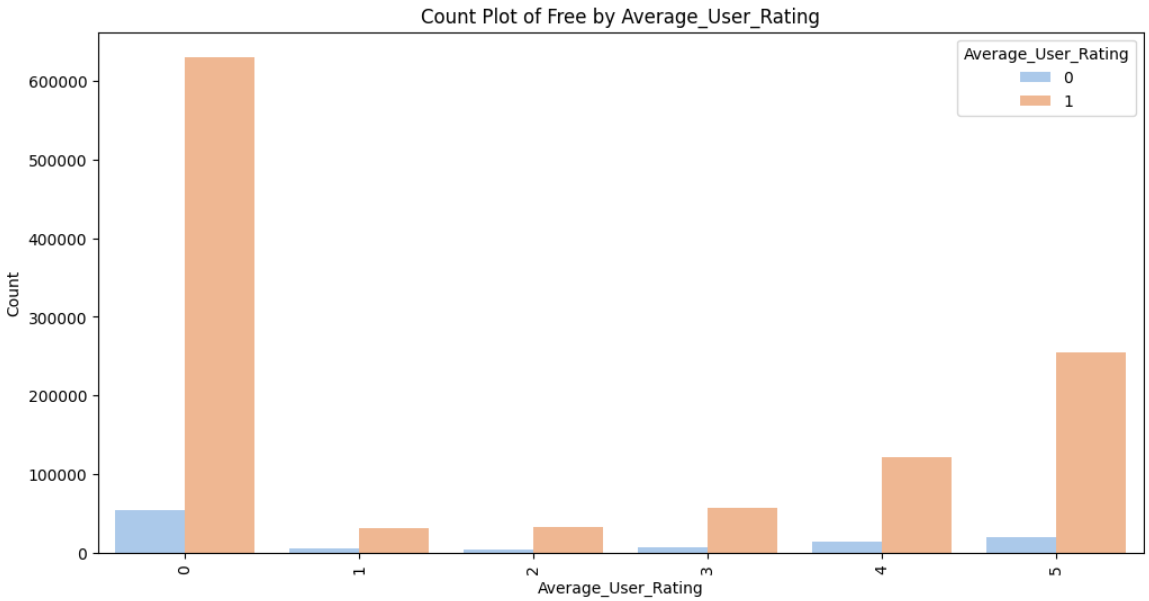
Years in which 'Education' primary genre apps were released: ['2017' '2018'
'2019' '2015' '2013' '2020' '2012' '2021' '2010' '2016'
'2011' '2014' '2009' '2008']

```
#rounding off the Average_User_Rating column
data['Average_User_Rating'] = data['Average_User_Rating'].round().astype(int)
```

```
data['Average_User_Rating'].unique()
array([0, 5, 3, 4, 2, 1])
```

- Count Plot of Free by Average_User_Rating

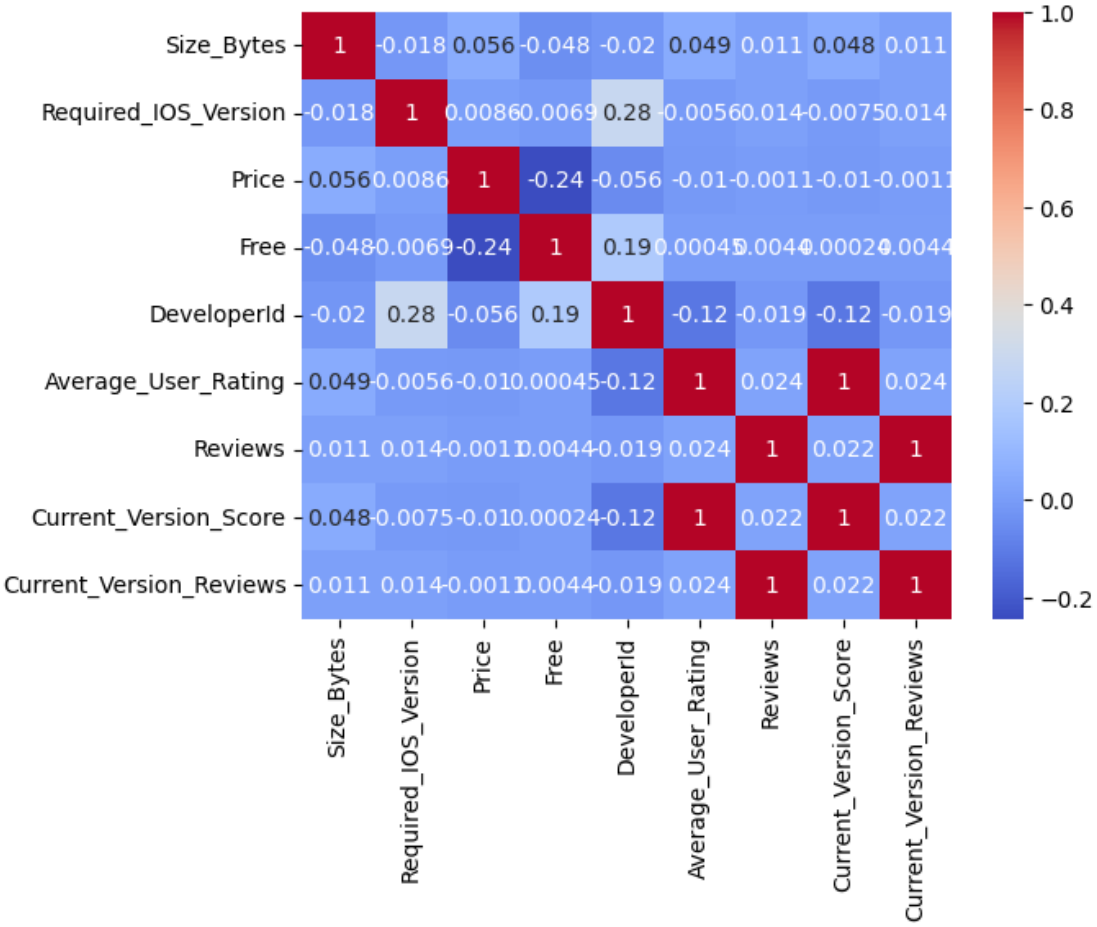
```
plt.figure(figsize=(12, 6))
sns.countplot(x='Average_User_Rating', hue='Free', data=data, palette='pastel')
plt.xlabel('Average_User_Rating')
plt.ylabel('Count')
plt.title('Count Plot of Free by Average_User_Rating')
plt.xticks(rotation=90)
plt.legend(title='Average_User_Rating', loc='upper right')
plt.show()
```



- User rating per year based on the price

  rating_per_price = data.groupby(['Average_User_Rating', 'Released_Year'])[['Price']].sum().reset_index()
  Rating_per_price

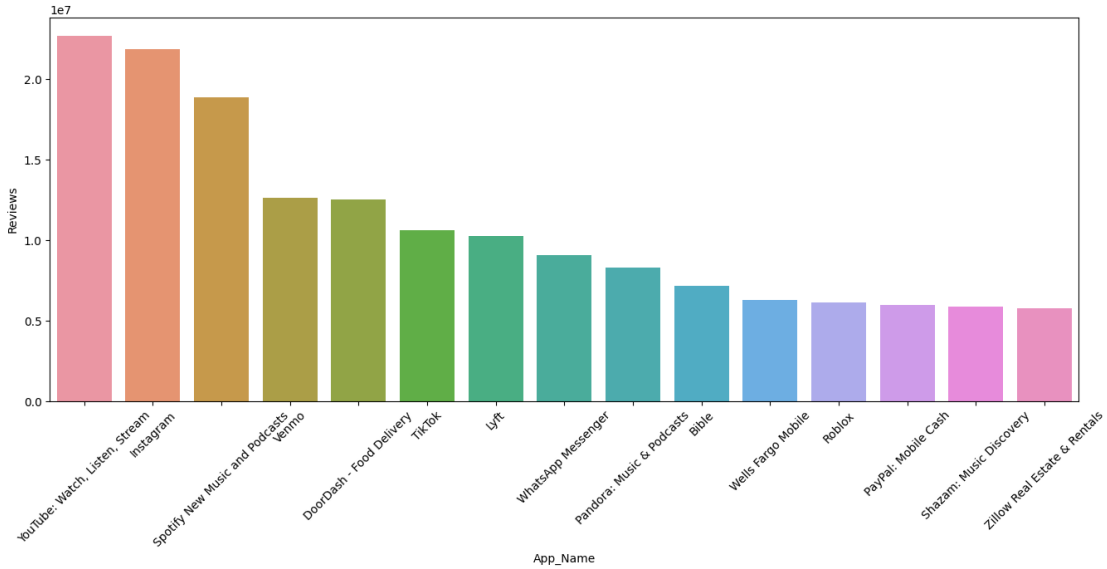
- Corelation plot of the numeric only
  corr = data.corr(numeric_only = True,)
  sns.heatmap(corr,cbar= True, cmap='coolwarm', annot=True)



- Top 10 Apps having highest reviews on AppleApp
  Top_reviews_apps = data[['App_Name', 'Reviews']].loc[data['Reviews'] >= 1000000].sort_values(by='Reviews', ascending=False).reset_index(drop=True).head(15) #reviews greater than 10L and top 25 data only
  Top_reviews_apps

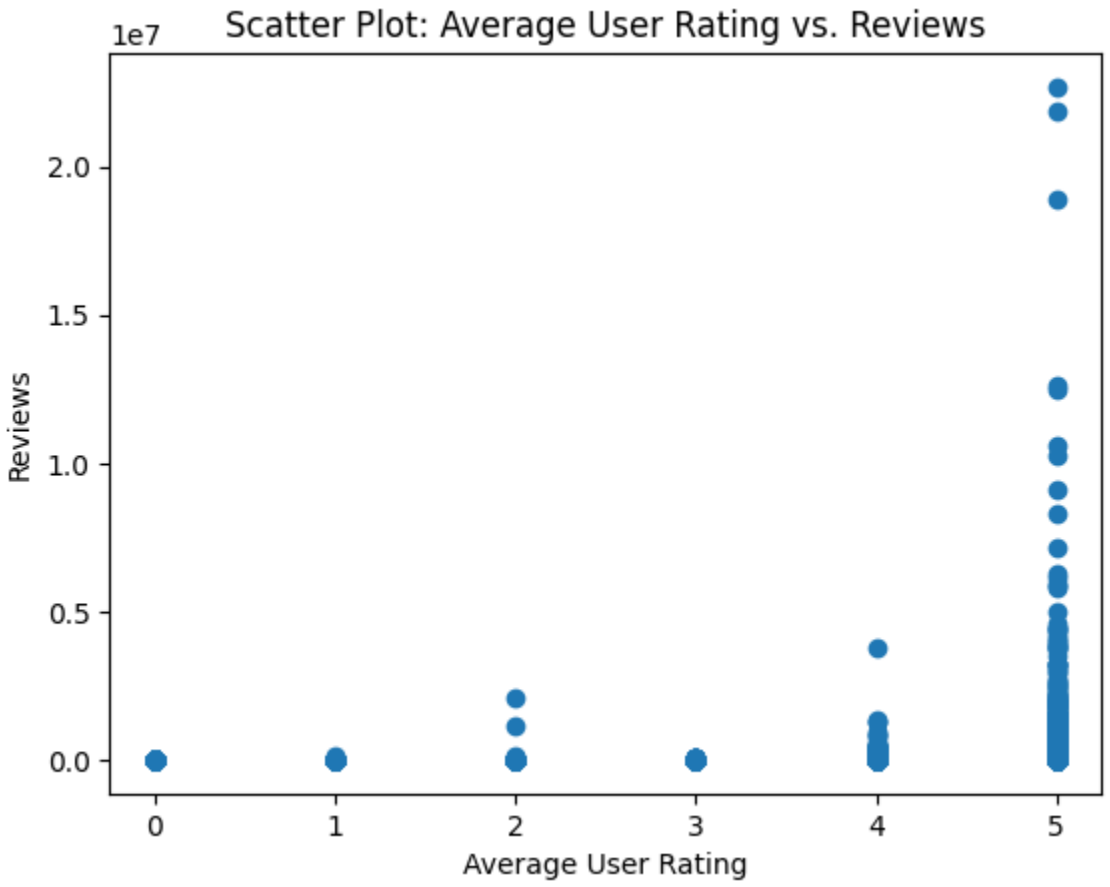
	App_Name	Reviews
0	YouTube: Watch, Listen, Stream	22685334
1	Instagram	21839585
2	Spotify New Music and Podcasts	18893225
3	Venmo	12634191
4	DoorDash - Food Delivery	12517538
5	TikTok	10598509
6	Lyft	10241777
7	WhatsApp Messenger	9090956
8	Pandora: Music & Podcasts	8297798
9	Bible	7181127
10	Wells Fargo Mobile	6275778
11	Roblox	6127588
12	PayPal: Mobile Cash	5962807
13	Shazam: Music Discovery	5895638
14	Zillow Real Estate & Rentals	5771436

  plt.figure(figsize=(16,6))
  sns.barplot(data = Top_reviews_apps, x = 'App_Name', y = 'Reviews')
  plt.xticks(rotation=45);



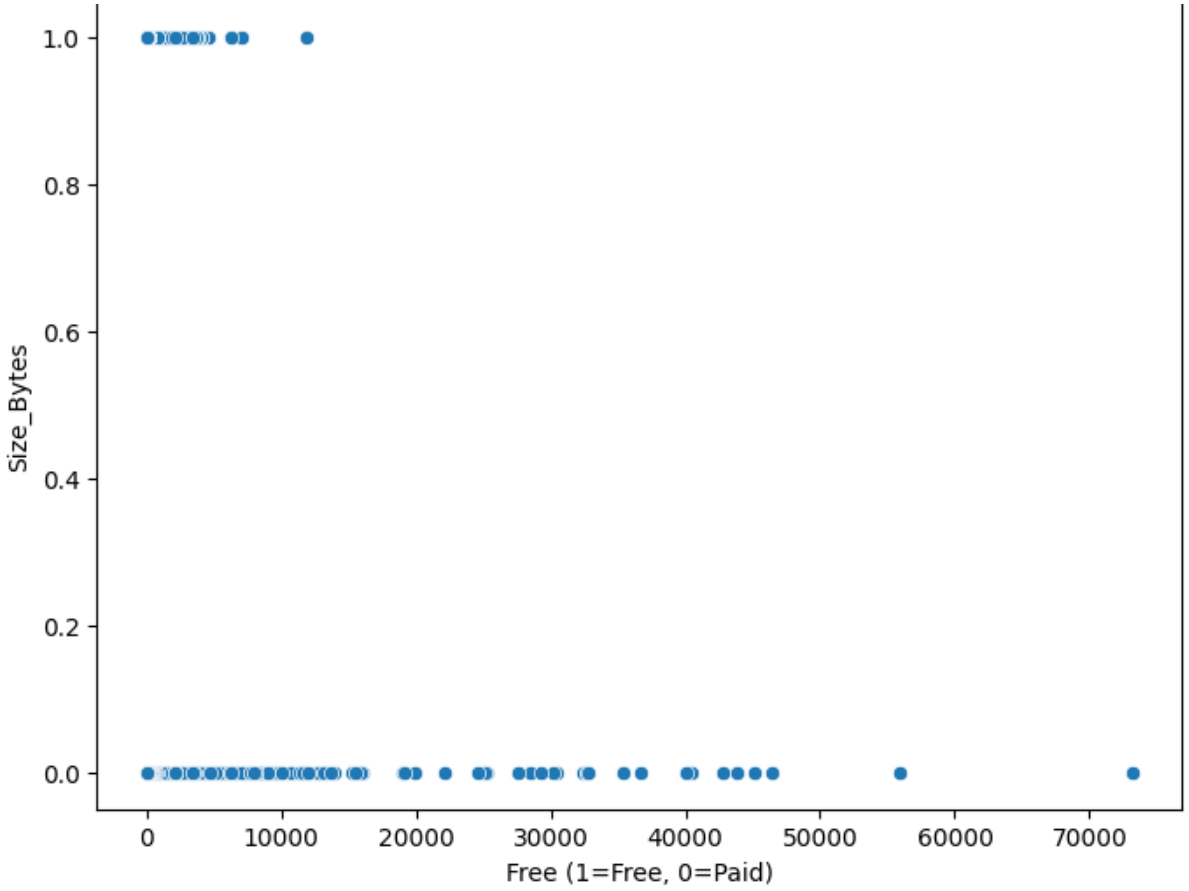
Youtube has the highest Review in the dataset.

- Scatter Plot: Average User Rating vs. Reviews
  plt.scatter(data['Average_User_Rating'], data['Reviews'])
  plt.xlabel('Average User Rating')
  plt.ylabel('Reviews')
  plt.title('Scatter Plot: Average User Rating vs. Reviews')
  plt.show()



- Scatter Plot of Size_Bytes for Free and Paid Apps

```
plt.figure(figsize=(8, 6))
sns.scatterplot(y='Free', x='Size_Bytes', data=data, palette='pastel')
plt.xlabel('Free (1=Free, 0=Paid)')
plt.ylabel('Size_Bytes')
plt.title('Scatter Plot of Size_Bytes for Free and Paid Apps')
plt.show()
```



MultiVariate Analysis

- Year and the top-5 Genre based on App Price

```
Rev_per_Genre = data.groupby(['Released_Year','Primary_Genre'
])[['Price']].sum()
print(Rev_per_Genre)

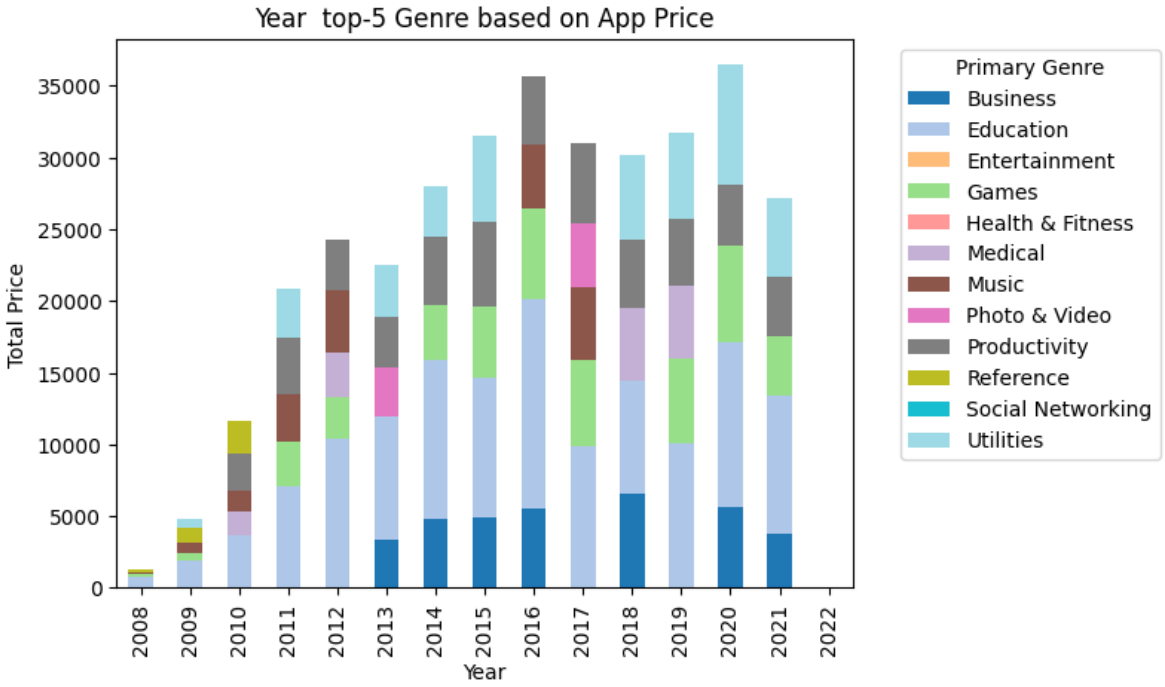
#Sorting values based on released year and price and grouping them according
to the Released year and taking the head as 5
top5_genres = (Rev_per_Genre
                .sort_values(['Released_Year', 'Price'], ascending=[True, False])
                .groupby('Released_Year')
                .head(5))

cmap = plt.get_cmap('tab20')

#summuation and plotting the each genre according to the price
top5_genres.groupby(['Released_Year',
'Primary_Genre'])['Price'].sum().unstack().plot(kind='bar', stacked=True, title=
"Year  top-5 Genre based on App Price", cmap=cmap)
plt.xlabel('Year')
plt.ylabel('Total Price')
plt.legend(title='Primary Genre', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.show()
```

```
                          Price
Released_Year Primary_Genre
2008          Book            5.980000
              Business       20.950000
              Education      779.990000
              Entertainment   45.860000
              Finance        49.840000
...                           ...
2022          Games           23.189814
              Health & Fitness  0.504126
              Productivity      0.504126
              Social Networking 0.504126
              Utilities         0.504126

[359 rows x 1 columns]
```




```
#Converting the Released year to numeric
data['Released_Year'] = pd.to_numeric(data['Released_Year'], errors='coerce')
non_numeric_years = data[pd.isna(data['Released_Year'])]

print(non_numeric_years)
print(data['Released_Year'].dtype)
```

Empty DataFrame
Columns: [App_Id, App_Name, Primary_Genre, Size_Bytes, Required_IOS_Version, Version, Price, Free, DeveloperId, Developer, Average_User_Rating, Reviews, Current_Version_Score, Current_Version_Reviews, ContentRating, Released_Year, Updated_Year] Index: [] int64

```
filtered_data = data[(data['Primary_Genre'] == 'Book') & (data['Released_Year'] == 2008)]
book_prices = filtered_data[['App_Name', 'Price']]
print(book_prices)
```

	App_Name	Price
5291	Festive Holiday Baking	0.99
28137	ShortBook	4.99

```
content_per_Genre = data.groupby(['Primary_Genre','ContentRating' ])[['Price']].sum()
```

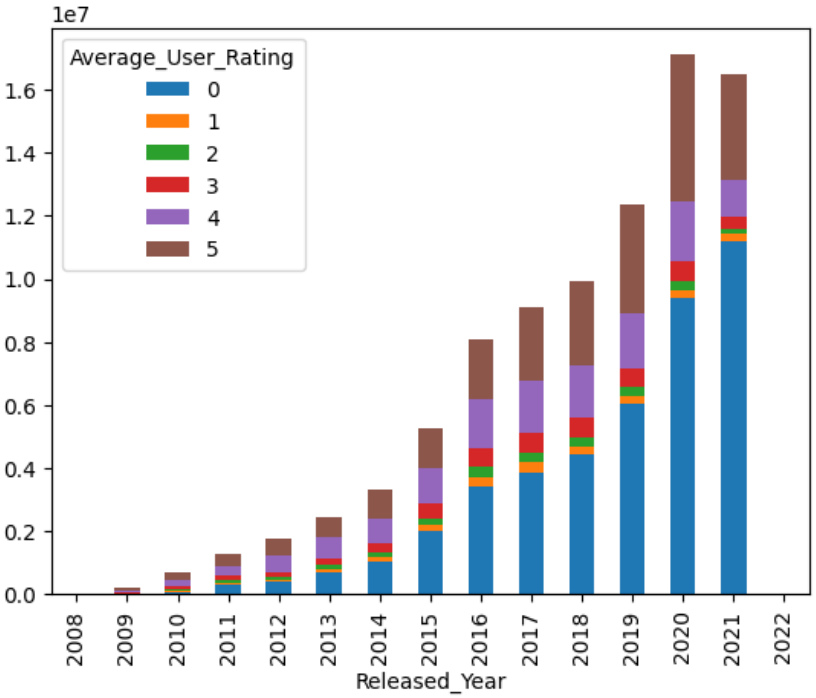
print(content_per_Genre)

```
                                Price
Primary_Genre ContentRating
Book          Adults          463.360000
              Children        127.640000
              Teens           460.280000
              Toddler        6130.090000
Business      Adults          1675.004126
...                                ...
Utilities     Toddler        51192.856506
Weather       Adults            27.880000
              Children           0.000000
              Teens             30.850000
              Toddler         2315.290000

[112 rows x 1 columns]
```

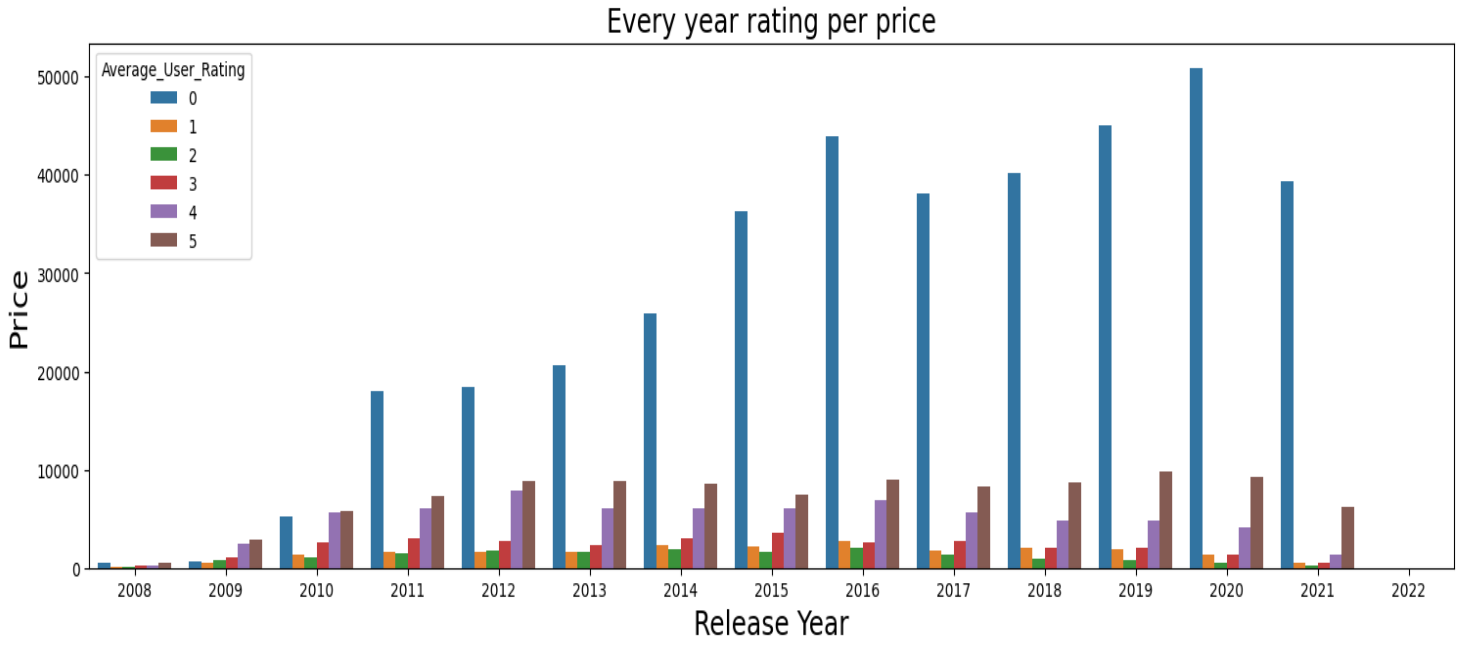
- Average user Rating and Released Year with the sum of Size Bytes

  #Grouping Average user Rating and Released Year with the sum of Size Bytes
  usr_Rat_MB = data.groupby(['Average_User_Rating', 'Released_Year'])[['Size_Bytes']].sum()
  usr_Rat_MB.groupby(['Released_Year', 'Average_User_Rating'])['Size_Bytes'].sum().unstack().plot(kind='bar', stacked=True)



- Released Year with the Price and considering the Average User Rating

```
plt.figure(figsize=(18,5))
sns.barplot(data = rating_per_price, x = 'Released_Year', y = 'Price', hue =
'Average_User_Rating')
plt.title('Every year rating per price',fontdict={'fontsize': 18})
plt.xlabel('Release Year', fontdict={'fontsize': 18})
plt.ylabel('Price', fontdict={'fontsize': 18})
```




```
large_reviews_threshold = 1000
large_reviews_apps = data[data['Reviews'] > large_reviews_threshold]

highly_rated_threshold = 4.5
highly_rated_apps = large_reviews_apps[large_reviews_apps['Average_User_Rating'] >=
highly_rated_threshold]

common_characteristics = highly_rated_apps[['App_Name', 'Primary_Genre', 'Price', 'Size_Bytes',
'Released_Year']]
print(common_characteristics)
```

```
                                App_Name     Primary_Genre  Price  \
10      AA Audio Companion for Alcoholics Anonymous              Book    0.0
11                    AA Big Book (Unofficial)              Book    0.0
19                 AA Speaker Tapes & 12 Steps  Health & Fitness    0.0
175                  Daily Bible Devotional              Book    0.0
219                      Game of the Gods             Games    0.0
```

```
...                                    ...             ...    ...
1229978              Super VPN - Best VPN Master        Utilities    0.0
1230259              Symbol Keyboard for Texting        Utilities    0.0
1230260            Symbol Keyboard-Character Pad        Utilities    0.0
1230261               Symbol Keypad for Texting         Utilities    0.0
1230372            SÐµÑ□urÐµÐ¥ VÐ N - Wifi Proxy        Utilities    0.0

        Size_Bytes  Released_Year
10       24.922852          2017
11       60.188477          2015
19       26.846680          2015
175      55.845703          2014
219      26.529297          2020
...            ...           ...
1229978  60.169922          2019
1230259  41.919922          2014
1230260  40.442383          2015
1230261  41.190430          2015
1230372  37.208984          2019

[17350 rows x 5 columns]
```

```
large_reviews_threshold = 1000
large_reviews_apps = data[data['Reviews'] > large_reviews_threshold]

highly_rated_threshold = 4.5
highly_rated_apps = large_reviews_apps[large_reviews_apps['Average_User_Rating'] >=
highly_rated_threshold]

common_characteristics ={
   'Primary_Genre': highly_rated_apps['Primary_Genre'].mode().values[0],
   'Price': highly_rated_apps['Price'].mode().values[0],
   'Size_Bytes': highly_rated_apps['Size_Bytes'].mode().values[0]
}

print(common_characteristics)
```

```
{'Primary_Genre': 'Games', 'Price': 0.0, 'Size_Bytes': 71.62915824316278}
```